JAVASERVERPAGE'S

RR
@
BMSCE

Purpose of JSP

- **Java Server Pages (JSP)** technology enables you to mix regular, static HTML with dynamically generated content from servlets.
- Write the regular HTML in the normal manner, using familiar Web-page-building tools.
- Then enclose the code for the dynamic parts in special tags, most of which start with **<%** and end with **%>**.

  For example, **<I><%= request.getParameter("title") %></I>**

- Java Server Pages offers several advantages over competing technologies such as ASP, PHP, or ColdFusion.

1. **That JSP is widely supported and thus doesn't lock you into a particular operating system or Web server and**
2. **That JSP gives you full access to servlet and Java technology for the dynamic part, rather than requiring you to use an unfamiliar and weaker special- purpose language.**

- The process of making Java Server Pages accessible on the Web is much simpler than that for servlets.
- Assuming you have a Web server that supports JSP, you give your file a .jsp extension and simply install it in any place could put a normal Web page:
  - no compiling,
  - no packages, and
  - no user CLASSPATH settings.

However, although your personal environment doesn't need any special settings, the server still has to be set up with access to the servlet and JSP class files and the Java compiler

How JSP pages are Invoked

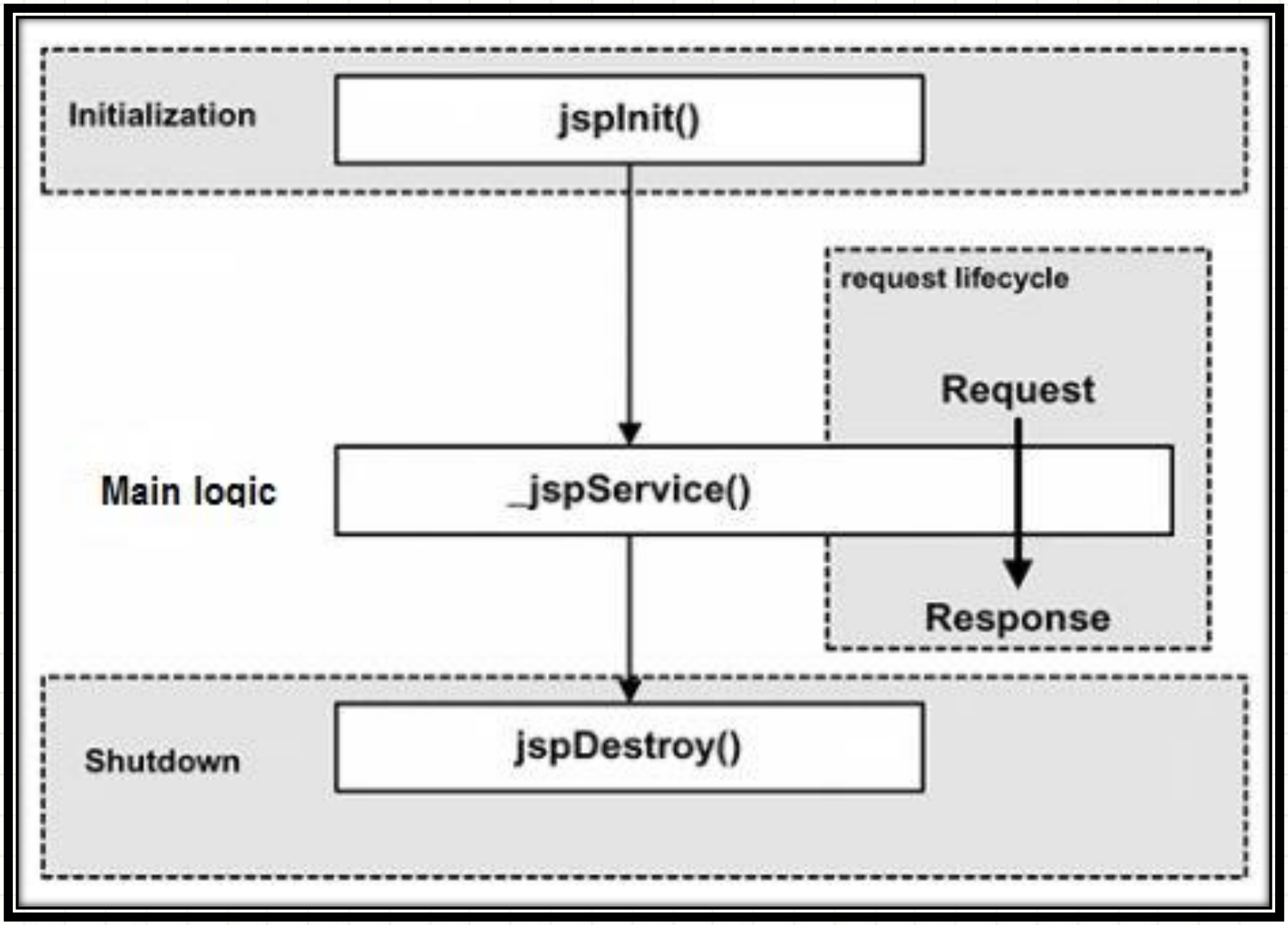
- **JSP** often looks like a regular HTML file than a servlet,
- The JSP page is automatically converted to a normal servlet, with the static HTML simply being printed to the output stream associated with the servlet's service method.
- This translation is done the first time the page is requested.
- To ensure that the first real user doesn't get a momentary delay when the JSP page is translated into a servlet and compiled, developers can simply request the page themselves after first installing it.

- If make an error in the dynamic portion of your JSP page, the system may not be able to properly translate it into a servlet.

Other than HTML, there are three main types of JSP constructs that embed in a page:

- ✓ **Scripting elements** let you specify Java code that will become part of the resultant servlet.
- ✓ **directives** let you control the overall structure of the servlet, and
- ✓ **actions** let you specify existing components that should be used and otherwise control the behavior of the JSP engine.

JSP Lifecycle



Differences between JSP & Servlet

JSP	Servlets
JSP is a webpage scripting language that can generate dynamic content.	Servlets are Java programs that are already compiled which also creates dynamic web content.
JSP run slower compared to Servlet as it takes compilation time to convert into Java Servlets.	Servlets run faster compared to JSP.
It's easier to code in JSP than in Java Servlets.	Its little much code to write here.
In MVC, jsp act as a view.	In MVC, servlet act as a controller.
JSP are generally preferred when there is not much processing of data required.	servlets are best for use when there is more processing and manipulation involved.
The advantage of JSP programming over servlets is that we can build custom tags which can directly call Java beans.	There is no such custom tag facility in servlets.
We can achieve functionality of JSP at client side by running JavaScript at client side.	There are no such methods for servlets.

Scripting Elements (1)

JSP scripting elements let you insert code into the servlet that will be generated from the JSP page. There are three forms:

1. Expressions of the form **<%= expression %> ,** which are evaluated and inserted into the servlet's output

2. Scriptlets of the form **<% code %>,** which are inserted into the servlet's _jspService method (called by service)

3. Declarations of the form **<%! code %> ,** which are inserted into the body of the servlet class, outside of any existing methods.

JSP Expressions

A JSP expression is used to insert values directly into the output.

It has the following form:

<%= Java Expression %>

- The expression is evaluated, converted to a string, and inserted in the page.
- This evaluation is performed at run time and has full access to information about the request.

For example, the following shows the date/time that the page was requested:

Current time: <%= new java.util.Date() %>

Scripting Elements (2)

Predefined Variables

To simplify expressions, use a number of predefined variables.The most important ones are:

- **request,** the HttpServletRequest
- **response,** the HttpServletResponse
- **session**, the HttpSession associated with the request
- **out**, the PrintWriter used to send output to the client

Here is an example:

Your hostname: <%= request.getRemoteHost() %>

Using Expressions as Attribute Values:-

JSP includes a number of elements that use XML syntax to specify various parameters.

<jsp:setProperty name="author“ property="firstName“ value="Marty" />

Most attributes require the value to be a fixed string enclosed in either single or double quotes . A few attributes , permit you to use a JSP expression that is computed at request time.

The value attribute of jsp:setProperty is one such example

<jsp:setProperty name="user“ property="id“ value=’<%= "UserID" + Math.random() %>’ />

Scripting Elements (3)

Attributes of JSP Expressions

Element Name Attribute Name(s)	*Element Name Attribute Name(s)*
jsp:setProperty	name or value
jsp:include	Page
jsp:forward	Page
jsp:param	Value

EXAMPLE

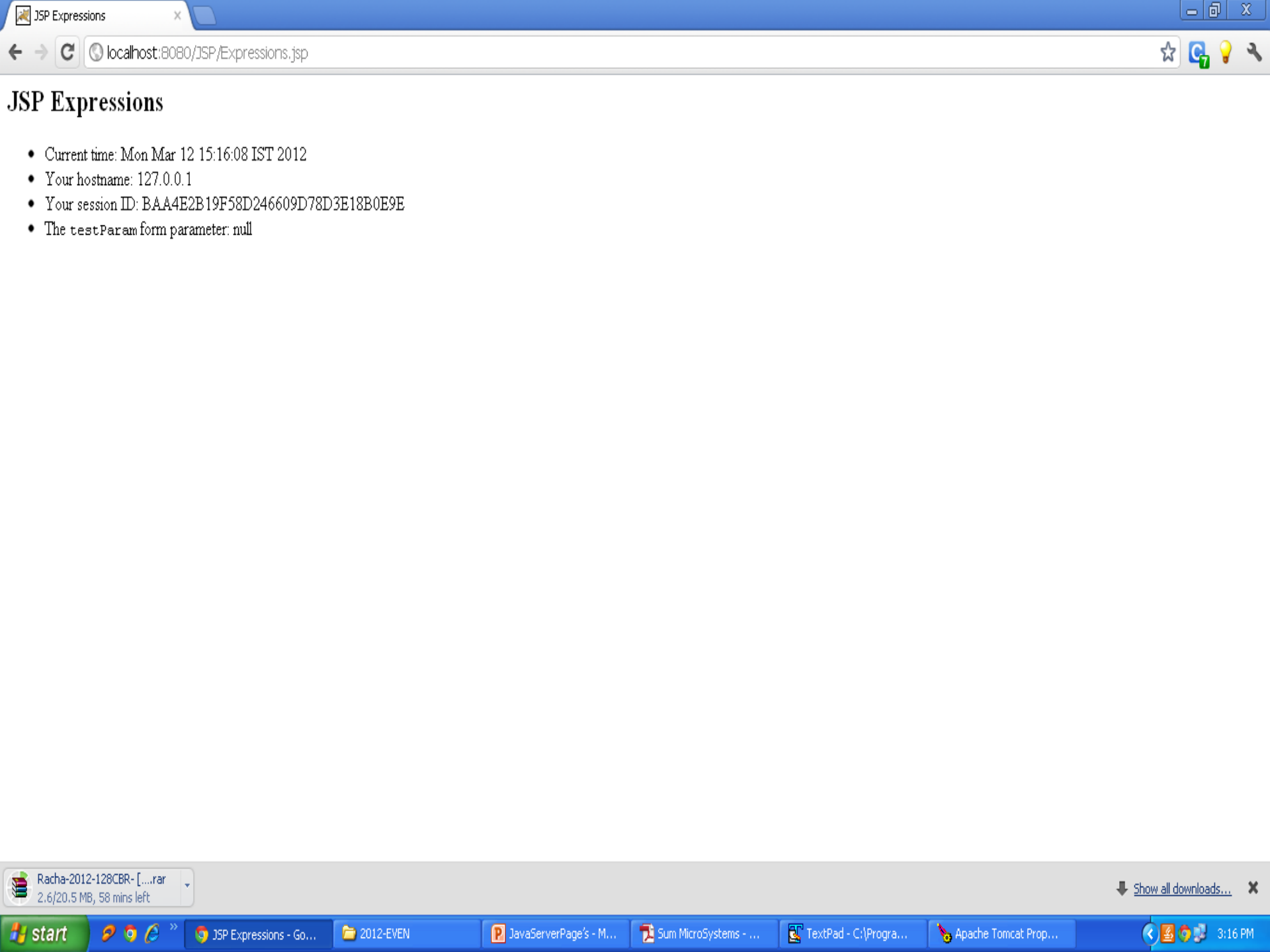
```
<HTML>
        <HEAD>
                <TITLE>JSP Expressions</TITLE>
        </HEAD>
        <BODY>

                <H2>JSP Expressions</H2>
                <UL>
                <LI>Current time: <%= new java.util.Date() %>
                <LI>Your hostname: <%= request.getRemoteHost() %>

                <LI>Your session ID: <%= session.getId() %>
                <LI>The <CODE>testParam</CODE> form parameter:

                <%= request.getParameter("testParam") %>

                </UL>
        </BODY>
</HTML>
```


JSP Expressions
localhost:8080/JSP/Expressions.jsp
JSP Expressions
Current time: Mon Mar 12 15:16:08 IST 2012
Your hostname: 127.0.0.1
Your session ID: BAA4E2B19F58D246609D78D3E18B0E9E
The testParam form parameter: null
Racha-2012-128CBR- [....rar
2.6/20.5 MB, 58 mins left
Show all downloads...
start
JSP Expressions - Go...
2012-EVEN
JavaServerPage's - M...
Sum MicroSystems - ...
TextPad - C:\Progra...
Apache Tomcat Prop...
3:16 PM

Scripting Elements (4)

JSP Scriptlets

JSP scriptlets used to insert arbitrary code into the servlet's _jspService method. Scriptlets have the following form:

<% Java Code %>

Scriptlets have access to the automatically defined variables as expressions (request, response, session, out).

For example, if you want output to appear in the resultant page, you would use the out variable, as in the following example.

```
<%
String queryData = request.getQueryString();
out.println("Attached GET data: " + queryData);
%>
```

In this instance, accomplished the same effect more easily by using the following JSP expression:

Attached GET data: <%= request.getQueryString() %>

Scripting Elements (5)

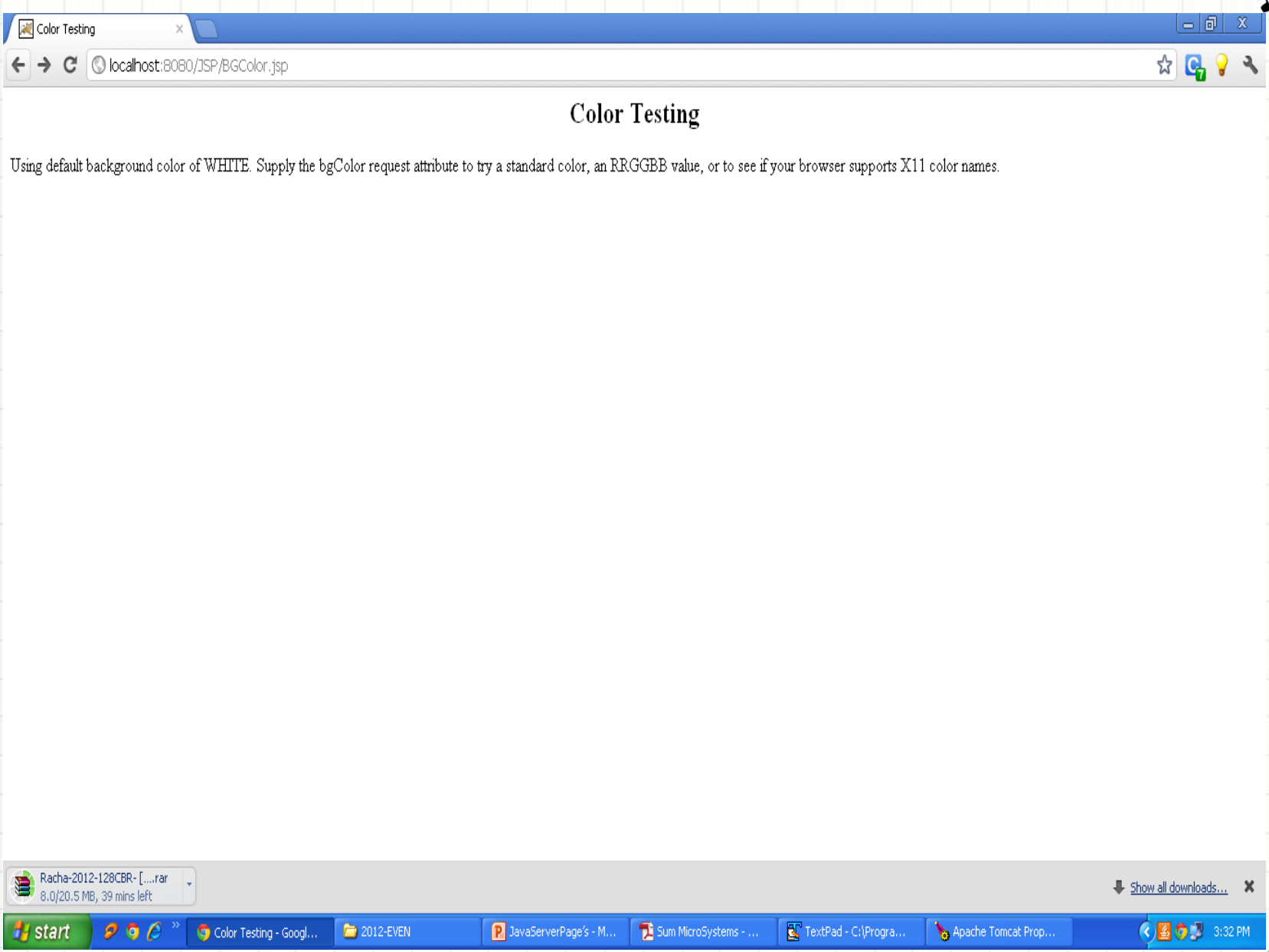
- **Scriptlets** can perform a number of tasks that cannot be accomplished with expressions alone.
- Tasks include setting response headers and status codes, invoking side effects such as writing to the server log or updating a database, or executing code that contains loops, conditionals, or other complex constructs.

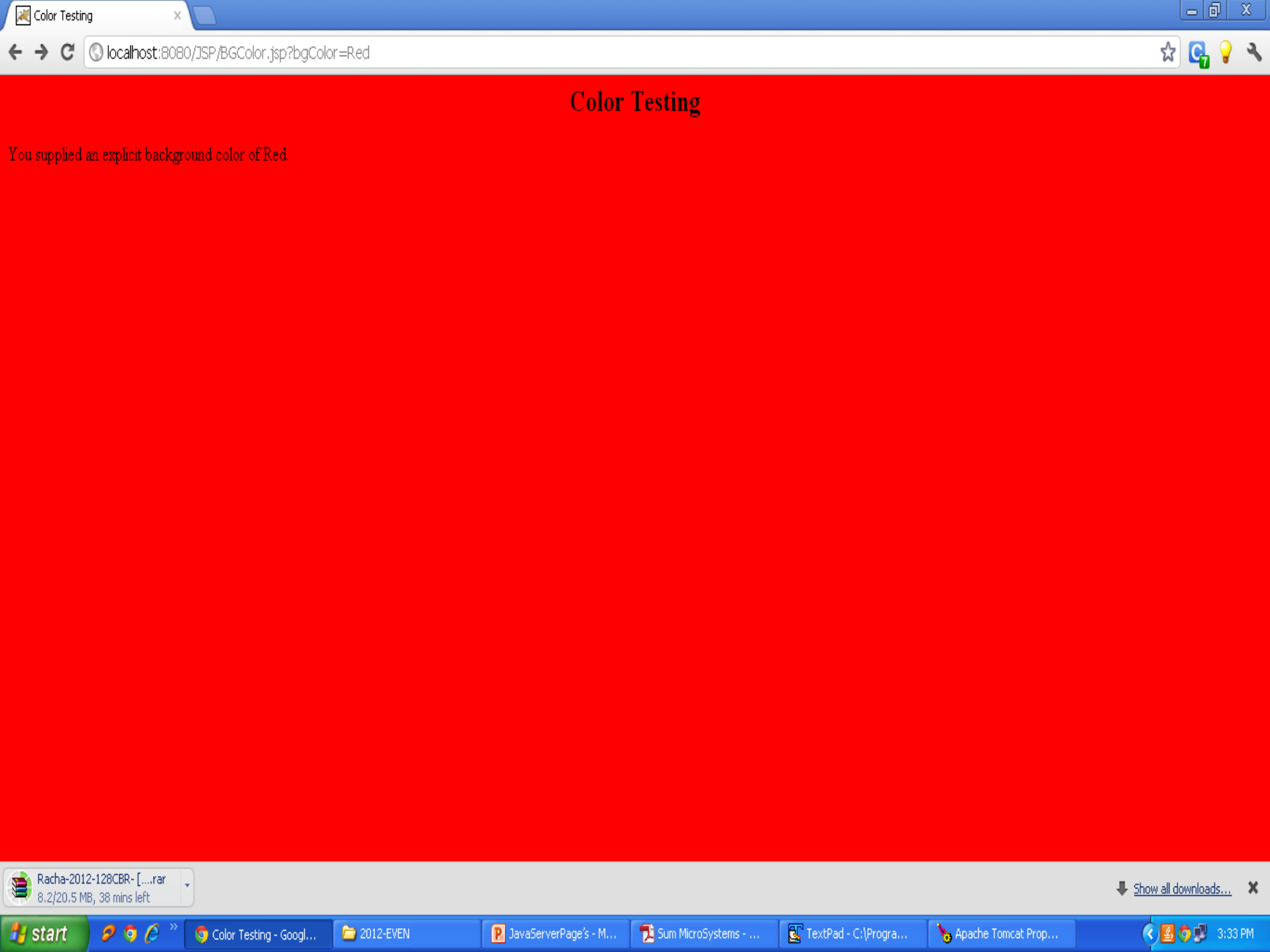
For instance, the following snippet specifies that the current page is sent to the client as plain text, not as HTML (which is the default).

<% response.setContentType("text/plain"); %>

As an example of executing code that is too complex for a JSP expression.

```
<HTML>
        <HEAD>
                <TITLE>Color Testing</TITLE>
        </HEAD>
<%      String bgColor = request.getParameter("bgColor");
        boolean hasExplicitColor;
        if (bgColor != null) {
        hasExplicitColor = true;
        } else {    hasExplicitColor = false;
                    bgColor = "WHITE";
                    }
%>      <BODY BGCOLOR="<%= bgColor %>">
        <H2 ALIGN="CENTER">Color Testing</H2>
<%      if (hasExplicitColor) {
        out.println("You supplied an explicit background color of " +
        bgColor + ".");
        } else {
        out.println("Using default background color of WHITE. " +
        "Supply the bgColor request attribute to try " +
        "a standard color, an RRGGBB value, or to see " +
        "if your browser supports X11 color names.");
        }
%>
</BODY>   </HTML>
```


Color Testing
localhost:8080/JSP/BGColor.jsp
Color Testing
Using default background color of WHITE. Supply the bgColor request attribute to try a standard color, an RRGGBB value, or to see if your browser supports X11 color names.
Racha-2012-128CBR- [....rar
8.0/20.5 MB, 39 mins left
Show all downloads...
start
Color Testing - Googl...
2012-EVEN
JavaServerPage's - M...
Sum MicroSystems - ...
TextPad - C:\Progra...
Apache Tomcat Prop...
3:32 PM


Color Testing
localhost:8080/JSP/BGColor.jsp?bgColor=Red
Color Testing
You supplied an explicit background color of Red.
Racha-2012-128CBR- [....rar
8.2/20.5 MB, 38 mins left
Show all downloads...
start
Color Testing - Googl...
2012-EVEN
JavaServerPage's - M...
Sum MicroSystems - ...
TextPad - C:\Progra...
Apache Tomcat Prop...
3:33 PM

Using Scriptlets to Make Parts of the JSP File Conditional

- The key approach is the fact that code inside a scriptlet gets inserted into the resultant servlet's _jspService method exactly as written, and any static HTML before or after a scriptlet gets converted to print statements.
- This means that scriptlets need not contain complete Java statements, and blocks left open can affect the static HTML or JSP outside of the scriptlets.

For example, consider the following JSP fragment containing mixed template text and scriptlets.

```
<% if (Math.random() < 0.5) { %>
Have a <B>nice</B> day!
<% } else { %>
Have a <B>Great</B> day!
<% } %>
```

When **converted to a servlet by the JSP engine**, this fragment will result in something similar to the following.

```
if (Math.random() < 0.5) {
out.println("Have a <B>nice</B> day!");
} else {
out.println("Have a <B>Great</B> day!");
}
```

JSP Declarations

A **JSP declaration** lets you define methods or fields that get inserted into the main body of the servlet class .

A declaration has the following form:

<%! Java Code %>

- Declarations do not generate any output, they are normally used in conjunction with JSP expressions or scriptlets.

For example, here is a JSP fragment that prints the number of times the current page has been requested since the server was booted.

Instance variables of a servlet are shared by multiple requests and accessCount does not have to be declared static below.

<%! private int accessCount = 0; %>

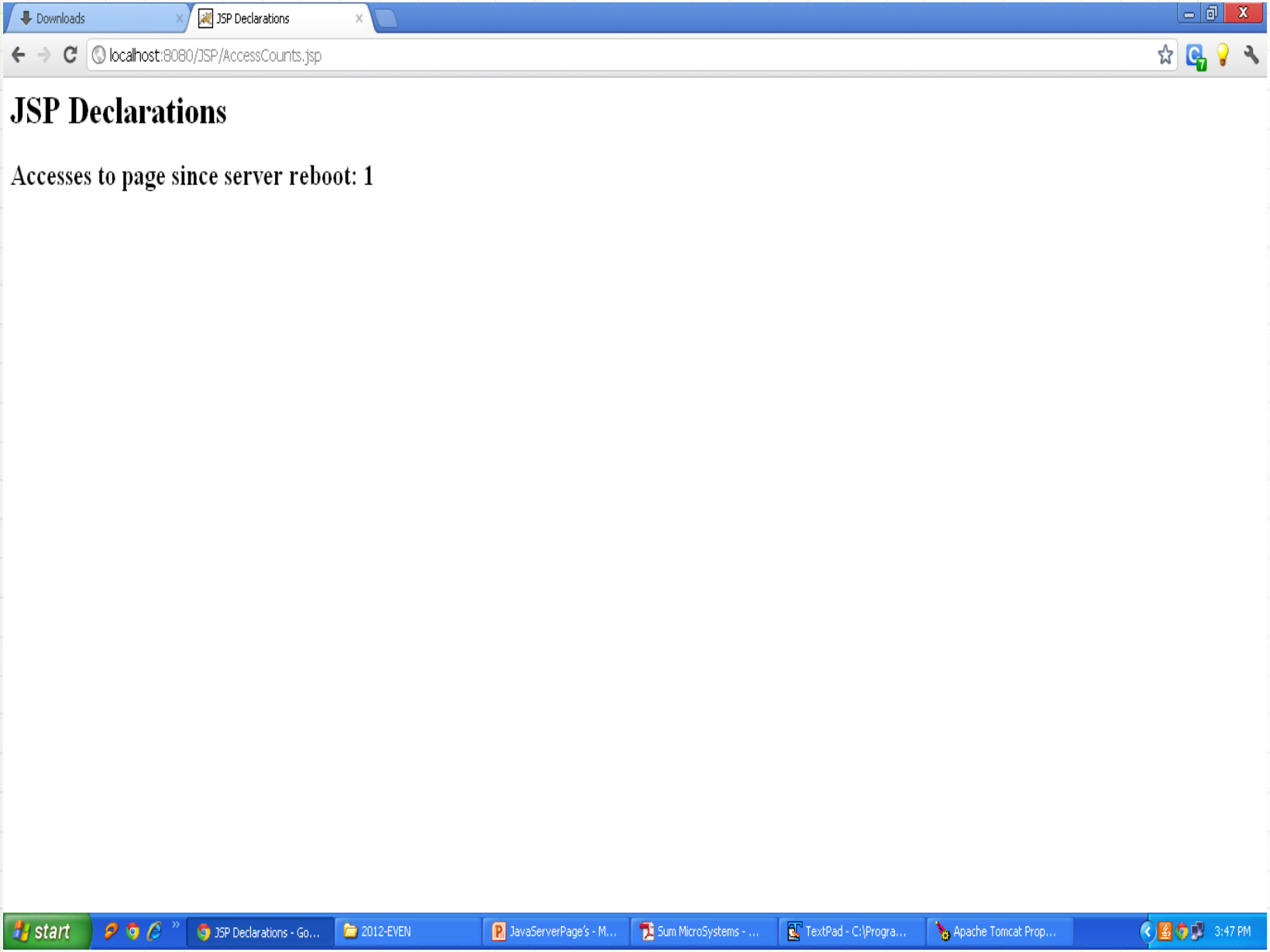
Accesses to page since server reboot:

<%= ++accessCount %>

```
<HTML>
        <HEAD>
                <TITLE>JSP Declarations</TITLE>

        </HEAD>
<BODY>
        <H1>JSP Declarations</H1>
                <%! private int accessCount = 0; %>
        <H2>Accesses to page since server reboot:
                <%= ++accessCount %></H2>
</BODY>

</HTML>
```


Downloads
JSP Declarations
localhost:8080/JSP/AccessCounts.jsp
JSP Declarations
Accesses to page since server reboot: 1
start
JSP Declarations - Go...
2012-EVEN
JavaServerPage's - M...
Sun MicroSystems - ...
TextPad - C:\Progra...
Apache Tomcat Prop...
3:47 PM


Downloads
JSP Declarations
localhost:8080/JSP/AccessCounts.jsp
JSP Declarations
Accesses to page since server reboot: 2
start
JSP Declarations - Go...
2012-EVEN
JavaServerPage's - M...
Sum MicroSystems - ...
TextPad - C:\Progra...
Apache Tomcat Prop...
3:48 PM

Predefined Variables(1)

- To simplify code in JSP expressions and scriptlets, **JSP** supplied with eight automatically defined variables, called **implicit objects**.

request

This variable is the HttpServletRequest associated with the request; it gives access to the request parameters, the request type (e.g., GET or POST), and the incoming HTTP headers (e.g., cookies).

out

This is the PrintWriter used to send output to the client. To make the response object useful, this is a buffered version of Print- Writer called JspWriter.

session

This variable is the HttpSession object associated with the request.Sessions are created automatically, so this variable is bound even if there is no incoming session reference.

Predefined Variables(2)

config

This variable is the ServletConfig object for this page.

pageContext

JSP introduced a new class called PageContext to give a single point of access to many of the page attributes and to provide a convenient place to store shared data. The pageContext variable stores the value of the PageContext object associated with the current page.

page

This variable is simply a synonym for this and is not very useful in the Java programming language. It was created as a place holder for the time when the scripting language could be something other than Java.

Predefined Variables(3)

response

This variable is the HttpServletResponse associated with the response to the client.

Application

This variable is the ServletContext as obtained via getServletConfig(). getContext(). Servlets and JSP pages can store persistent data in the ServletContext object rather than in instance variables.

ServletContext has setAttribute and getAttribute methods use to store arbitrary data associated with specified keys.

JSP page Directive

Purpose of the Directive

In **JSP**, there are three types of directives: **page, include, and taglib.**

- The **page directive** control the structure of the servlet by importing classes, customizing the servlet superclass, setting the content type.
- A **page directive** can be placed anywhere within the document;
- The **second directive, include**, insert a file into the servlet class at the time the JSP file is translated into a servlet.
- An **include directive** should be placed in the document at the point at which you want the file to be inserted;
- JSP 1.1 introduces a **third directive, taglib,** which can be used to define custom markup tags;

Page Directive

The **page directive** can define one or more of the following case-sensitive attributes: import, contentType, isThreadSafe, session, buffer, autoflush, extends, info, errorPage, isErrorPage, and language.

- **Import.**

- The import attribute of the page directive specify the packages that should be imported by the servlet into which the JSP page gets translated.
- If you don't explicitly specify any classes to import, the servlet imports java.lang.*, javax.servlet.*, javax.servlet.jsp.*, javax.servlet. http.*, and possibly some number of server-specific entries.

Use of the import attribute takes one of the following two forms:

1. ***<%@ page import="package.class" %>***
2. ***<%@ page import="package.class1,...,package.classN" %>***

For example, the following directive signifies that all classes in the java.util package should be available to use without explicit package identifiers.

<%@ page import="java.util.*" %>

- The import attribute is the only page attribute that is allowed to appear multiple times within the same document.
- page directives can appear anywhere within the document,
- It is traditional to place import statements either near the top of the document or just before the first place that the referenced package is used.

PAGE DIRECTIVE - ATTRIBUTE

Attribute	Purpose
buffer	Specifies a buffering model for the output stream
autoFlush	Controls the behavior of the servlet output buffer.
contentType	Defines the character encoding scheme
errorPage	Defines the URL of another JSP that reports on Java unchecked runtime exceptions.
isErrorPage	Indicates if this JSP page is a URL specified by another JSP page's errorPage attribute.
extends	Specifies a superclass that the generated servlet must extend
import	Specifies a list of packages or classes for use in the JSP as the Java import statement does for Java classes.
info	Defines a string that can be accessed with the servlet's getServletInfo() method.
isThreadSafe	Defines the threading model for the generated servlet.
language	Defines the programming language used in the JSP page.
session	Specifies whether or not the JSP page participates in HTTP sessions.
isELIgnored	Specifies whether or not EL expression within the JSP page will be ignored.
isScriptingEnabled	Determines if scripting elements are allowed for use.

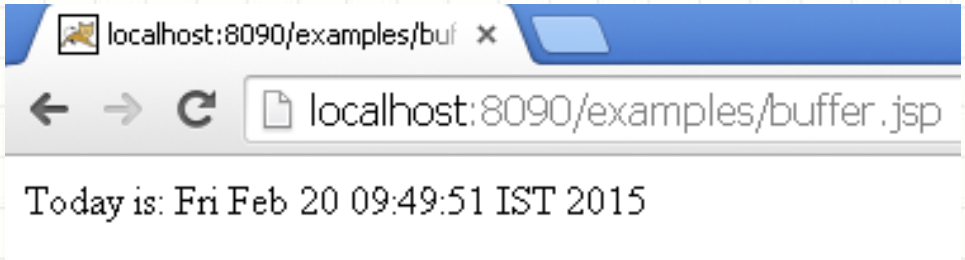
buffer

- The buffer attribute sets the buffer size in kilobytes to handle output generated by the JSP page.The default size of the buffer is 8Kb.

```
<html>
<body>

<%@ page buffer="16kb" %>
Today is: <%= new java.util.Date() %>

</body>
</html>
```



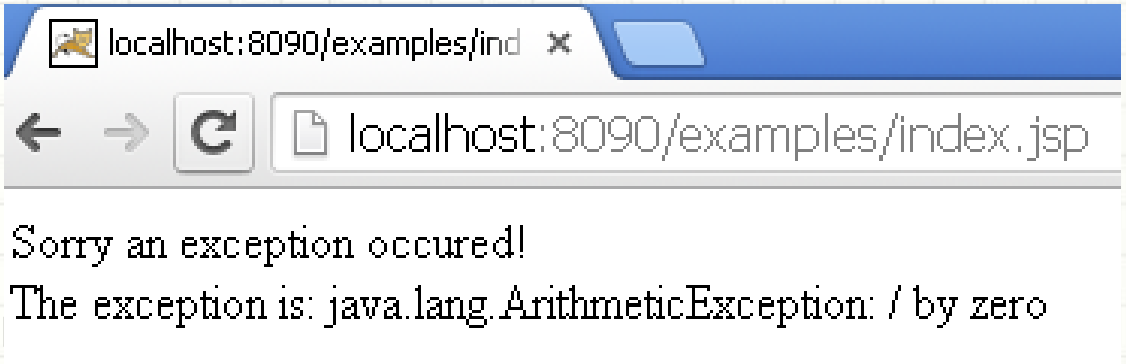
errorpage

Index.jsp

```
<html>
<body>
 <%@ page errorPage="myerrorpage.jsp" %>
 <%= 100/0 %>
</body>
</html>
```

myerrorpage.jsp

```
<html>
<body>
 <%@ page isErrorPage="true"
  Sorry an exception occured!<
The exception is: <%= exception %>
 </body>
</html>
```



include Directive

- The **include** directive is used to includes a file during the translation phase.
- This directive tells the container to merge the content of other external files with the current JSP during the translation phase.
- You may code include directives anywhere in your JSP page.

The general usage form of this directive is as follows:

<%@ include file="relative url" >

The filename in the include directive is actually a relative URL.

- If you just specify a filename with no associated path, the JSP compiler assumes that the file is in the same directory as your JSP.

footer.jsp

```
<br/><br/>
<center>
<p>Copyright © 2010</p>
</center>
</body>
</html>
```

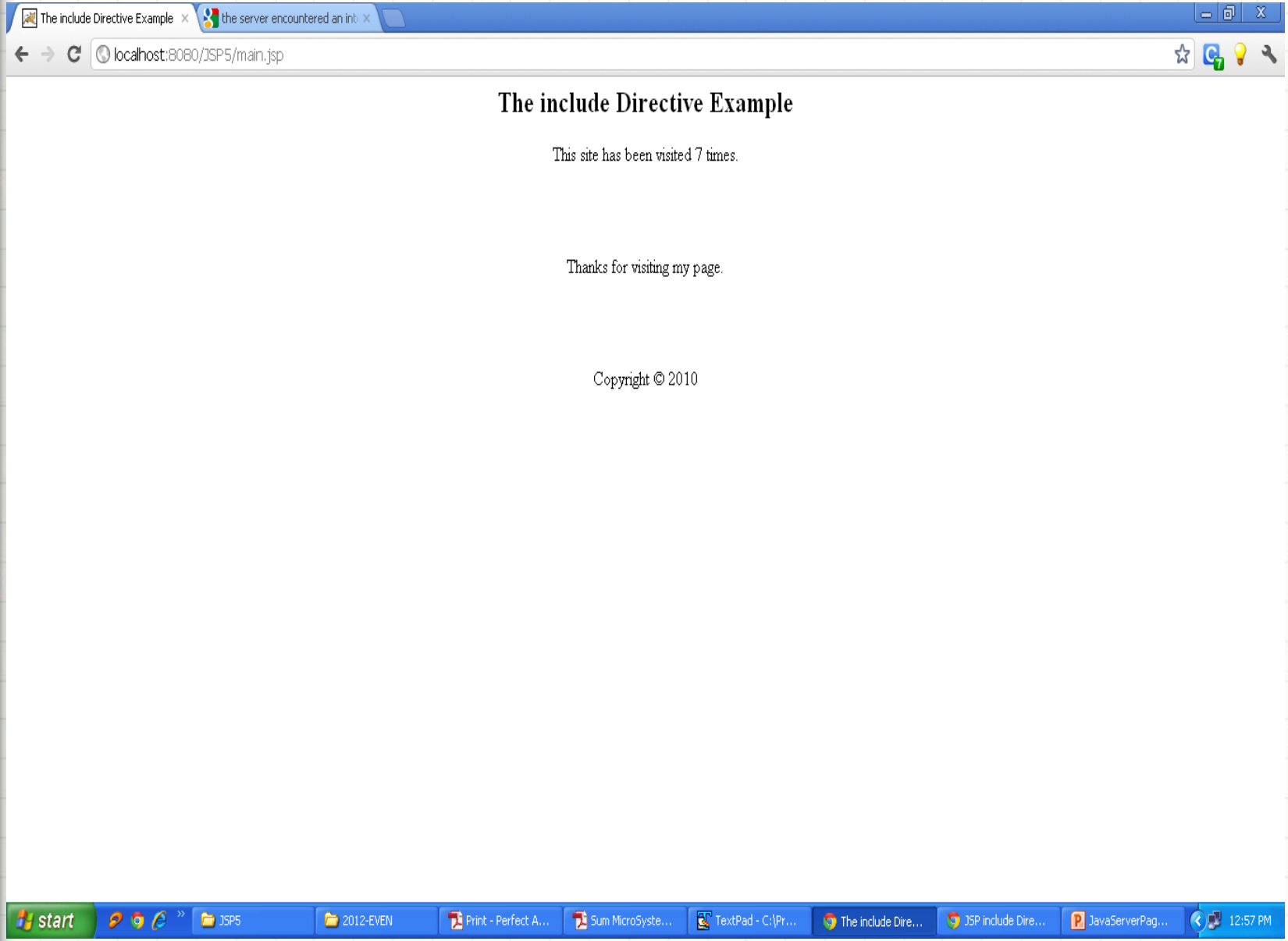
include Directive (Cont..)

header.jsp

```
<%!  int pageCount = 0;
 void addCount() {
   pageCount++;
 }
%>
<% addCount(); %>
<html>
<head>
<title>The include Directive Example</title>
</head>
<body>
<center>
<h2>The include Directive Example</h2>
<p>This site has been visited <%= pageCount %> times.</p>
</center>
<br/><br/>
```

Main.jsp

```
<%@ include file="header.jsp" %>
<center>
<p>Thanks for visiting my page.</p>
</center>
<%@ include file="footer.jsp" %>
```


The include Directive Example
the server encountered an int
localhost:8080/JSP5/main.jsp
The include Directive Example
This site has been visited 7 times.
Thanks for visiting my page.
Copyright © 2010
start
JSP5
2012-EVEN
Print - Perfect A...
Sum MicroSyste...
TextPad - C:\Pr...
The include Dire...
JSP include Dire...
JavaServerPag...
12:57 PM

INCLUDING FILES AT REQUEST TIME

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN">
<HTML>
<HEAD>
<TITLE>What's New</TITLE>
<LINK REL=STYLESHEET
HREF="JSP-Styles.css"
TYPE="text/css">
</HEAD>
<BODY>
<CENTER>
<TABLE BORDER=5>
<TR><TH CLASS="TITLE">
What's New at JspNews.com</TABLE>
</CENTER>
<P>
Here is a summary of our four most recent news stories:
<OL>
<LI><jsp:include page="news/Item1.html" flush="true" />
<LI><jsp:include page="news/Item2.html" flush="true" />
<LI><jsp:include page="news/Item3.html" flush="true" />
<LI><jsp:include page="news/Item4.html" flush="true" />
</OL>
</BODY>
</HTML>
```

<B>Bill Gates acts humble.</B> In a startling and unexpected development, Microsoft big wig Bill Gates put on an open act of humility yesterday.
<A HREF="http://www.microsoft.com/Never.html">More details...</A>

<B>Scott McNealy acts serious.</B> In an unexpected twist, wisecracking Sun head Scott McNealy was sober and subdued at yesterday's meeting.
<A HREF="http://www.sun.com/Imposter.html">More details...</A>

<B>Sportscaster uses "literally" correctly.</B> In an apparent slip of the tongue, a popular television commentator was heard to use the word "literally" when he did <I>not</I> mean "figuratively."
<A HREF="http://www.espn.com/Slip.html">More details...</A>

<B>Larry Ellison acts conciliatory.</B> Catching his competitors off guard yesterday, Oracle prez Larry Ellison referred to his rivals in friendly and respectful terms.
<A HREF="http://www.oracle.com/Mistake.html">More details...</A>


What's New
localhost:8080/JSP6/WhatsNew.jsp
What's New at JspNews.com
Here is a summary of our four most recent news stories:
1. Bill Gates acts humble. In a startling and unexpected development, Microsoft big wig Bill Gates put on an open act of humility yesterday. More details...
2. Scott McNealy acts serious. In an unexpected twist, wisecracking Sun head Scott McNealy was sober and subdued at yesterday's meeting. More details...
3. Sportscaster uses "literally" correctly. In an apparent slip of the tongue, a popular television commentator was heard to use the word "literally" when he did not mean "figuratively." More details...
4. Larry Ellison acts conciliatory. Catching his competitors off guard yesterday, Oracle prez Larry Ellison referred to his rivals in friendly and respectful terms. More details...
start
Piriform CCleaner
news
JavaServerPage's - M...
Sun MicroSystems - ...
TextPad - C:\Progra...
What's New - Google ...
9:00 AM

INCLUDING APPLETS FOR THE JAVA PLUG-IN

The jsp:plugin Element

- The simplest way to use jsp:plugin is to supply four attributes:

type, code, width, and height.

you could replace

```
<APPLET CODE="MyApplet.class“   WIDTH=475 HEIGHT=350>
</APPLET>
```

with

```
<jsp:plugin type="applet“  code="MyApplet.class“  width="475"  height="350">
</jsp:plugin>
```

- **type**

For applets, this attribute should have a value of applet. The Java Plug-In also permits embed JavaBeans elements in Web pages. Use a value of bean in such a case.

- **code**

This attribute is used identically to the CODE attribute of APPLET, specifying the top-level applet class file that extends Applet or JApplet.

- **width**

This attribute is used identically to the WIDTH attribute of APPLET, specifying the width in pixels to be reserved for the applet .

- **height**

This attribute is used identically to the HEIGHT attribute of APPLET, specifying the height in pixels to be reserved for the applet.

```
public class ButtonMoveApplet extends Applet {
 Button move;
 Random r;
 public void init() {
 setLayout(null);
 move = new Button("Click me");
 add(move);
 move.reshape(10,10,70,30);
 r = new Random();
 setBackground(Color.red);
 setForeground(Color.yellow);
 }
 public void paint(Graphics g){
 g.drawString("Welcome JSP-Applet",100,100);
 }
 public boolean action(Event evt, Object whatAction) {
if (!(evt.target instanceof Button))
return false;
  String buttonLabel = (String)whatAction;
if (buttonLabel == "Click me") {
move.reshape(Math.abs(r.nextInt())%(size().width-70),
Math.abs(r.nextInt())%(size().height-30),70,30);
repaint();
 }    return true;     }   }
```

```
<%@ page language="java" %>
<html>
        <head>
        <title>Welcome JSP-Applet Page</title>
        </head>

    <body>
        <jsp:plugin type="applet" code="ButtonMoveApplet.class"
         width="400" height="400">

                <jsp:fallback>
                        <p>Unable to load applet</p>
                </jsp:fallback>
        </jsp:plugin>
    </body>
</html>
```


Welcome JSP-Applet Page
localhost:8080/JavaApplet/Applet.jsp
Click me
Welcome JSP-Applet


Welcome JSP-Applet Page
localhost:8080/JavaApplet/Applet.jsp
Welcome JSP-Applet
Click me

JSP Actions

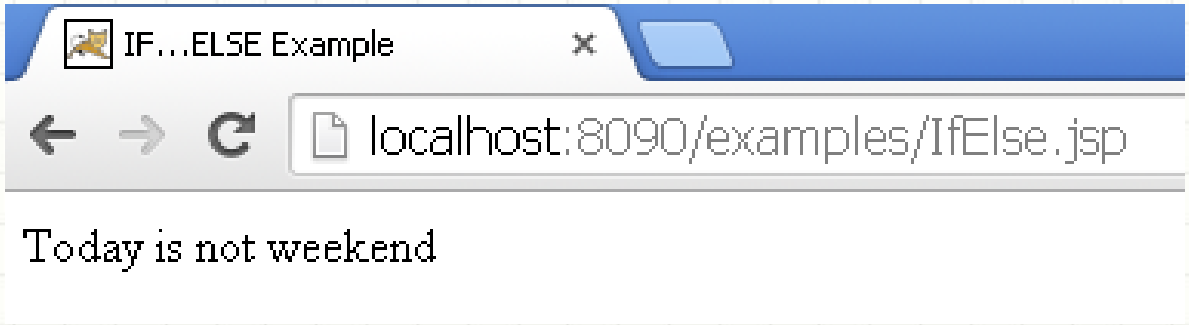
- JSP actions use constructs in XML syntax to control the behavior of the servlet engine. You can dynamically insert a file, reuse JavaBeans components, forward the user to another page, or generate HTML for the Java plugin.
- There is only one syntax for the Action element, as it conforms to the XML standard:

<jsp:action_name attribute="value" />

Syntax	Description
jsp:include	Includes a file at the time the page is requested
jsp:useBean	Finds or instantiates a JavaBean
jsp:setProperty	Sets the property of a JavaBean
jsp:getProperty	Inserts the property of a JavaBean into the output
jsp:forward	Forwards the requester to a new page
jsp:plugin	Generates browser-specific code that makes an OBJECT or EMBED tag for the Java plugin
jsp:element	Defines XML elements dynamically.

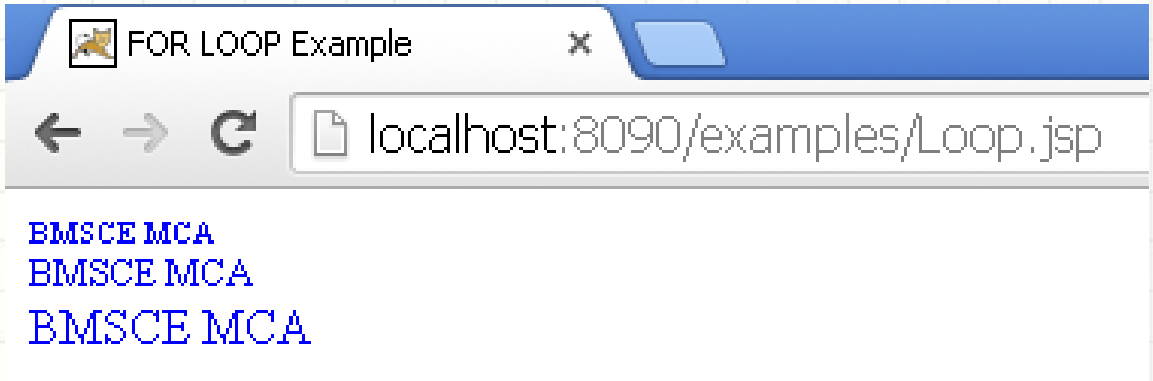
Control Statements(1)

```
<%! int day = 3; %>
<html>
<head><title>IF...ELSE Example</title></head>
<body>
<% if (day == 1 | day == 7) { %>
    <p> Today is weekend</p>
<% } else { %>
    <p> Today is not weekend</p>
<% } %>
</body>
</html>
```



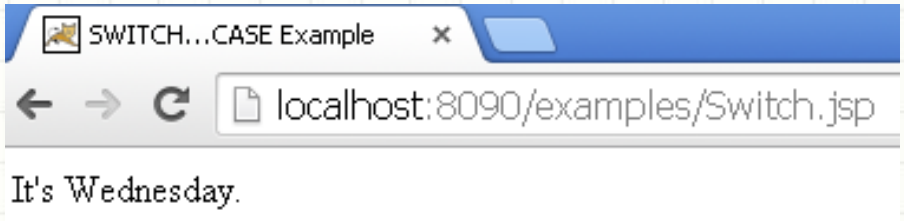
Control Statements(2)

```
<%! int fontSize; %>
<html>
<head><title>FOR LOOP Example</title></head>
<body>
<%for ( fontSize = 1; fontSize <= 3; fontSize++){ %>
  <font color="blue" size="<%= fontSize %>">
   BMSCE MCA
  </font><br />
<%}%>
</body>
</html>
```



Control Statements(3)

```
<%! int day = 3; %>
<html>  <head><title>SWITCH...CASE Example</title></head>
<body>
<%          switch(day) {
case 0:
  out.println("It\'s Sunday.");
  break;
case 1:
  out.println("It\'s Monday.");
  break;
case 2:
  out.println("It\'s Tuesday.");
  break;
case 3:
  out.println("It\'s Wednesday.");
  break;
case 4:
  out.println("It\'s Thursday.");
  break;
case 5:
  out.println("It\'s Friday.");
  break;
default:
  out.println("It's Saturday.");
}
%>
</body>
</html>
```



USING JAVABEANS WITH JSP

Why Java beans

The **JavaBeans API** provides a standard format for Java classes.

- Visual manipulation tools and other programs can automatically discover information about classes that follow this format and
- Create and manipulate the classes without the user having to explicitly write any code.

Three simple points about beans:

1. A bean class must have a zero-argument constructor. The empty constructor will be called when JSP elements create beans.

2. A bean class should have no public instance variables .

3. *Persistent values should be accessed through methods called getXxx and setXxx*.

Basic Bean Use

The jsp:useBean action lets you load a bean to be used in the JSP page.

Beans provide a very useful capability because they let you exploit the reusability of Java classes without sacrificing the convenience that JSP adds over servlets alone.

Java Bean Properties

- A JavaBean property is a named attribute that can be accessed by the user of the object.
- The attribute can be of any Java data type, including classes that you define.
- A JavaBean property may be read, write, read only, or write only.
- JavaBean properties are accessed through two methods in the JavaBean's implementation class:

Method Description

- **getPropertyName()** For example, if property name is firstName, your method name would be getFirstName() to read that property. This method is called accessor.

- **setPropertyName()** For example, if property name is firstName, your method name would be setFirstName() to write that property. This method is called mutator.

- A read-only attribute will have only a getPropertyName() method, and a write-only attribute will have only a setPropertyName() method.

Bean Scope

Scope has four possible settings:

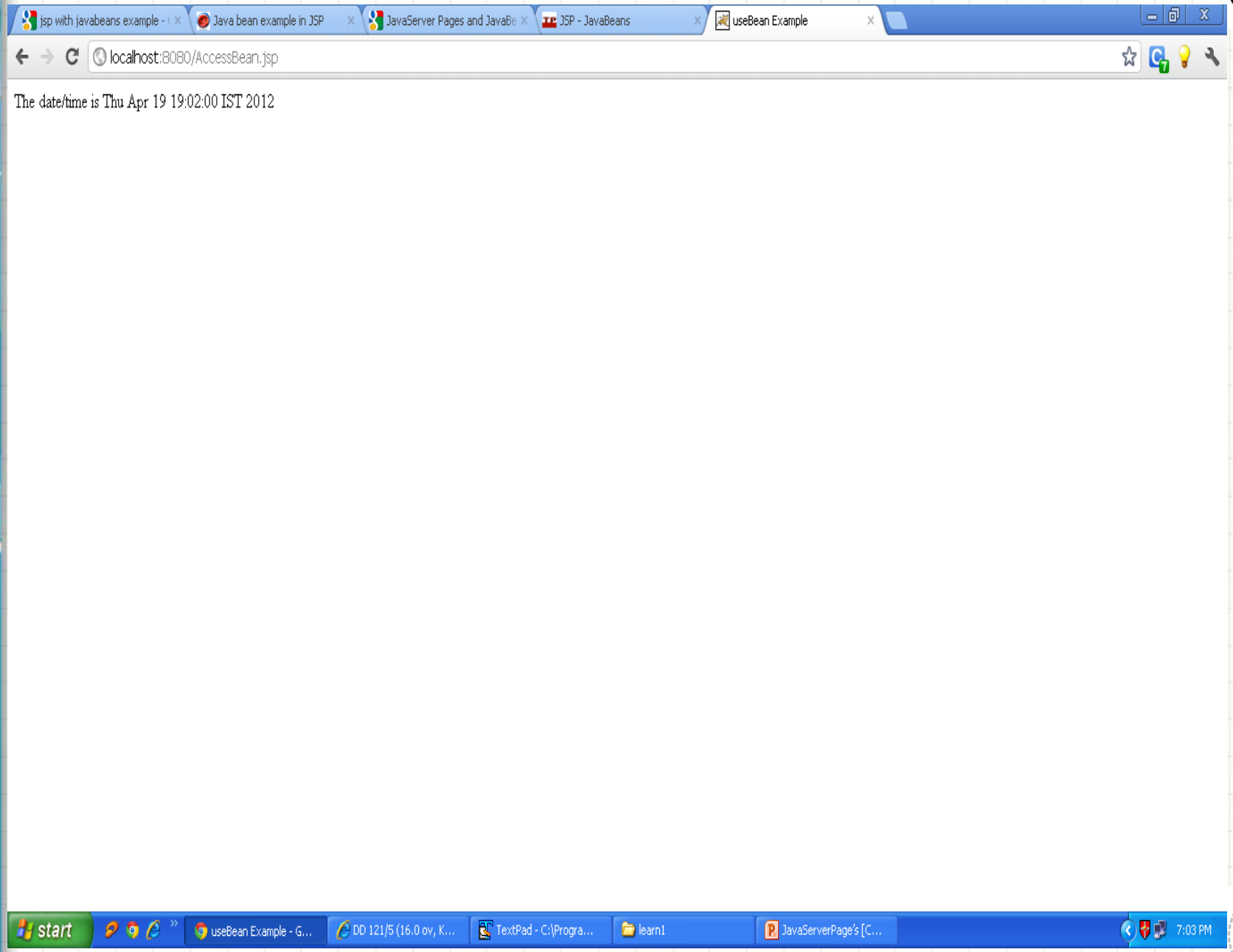
1. **Page -** Bean exists for execution of that page only
2. **Request -** Like page, but survives any forward or include requests
3. **Session**
   - The Bean exists for multiple requests within the web application, from a particular web browser instance
   - Used for shopping baskets etc.
4. **Application**
   - The Bean exists for all requests from all users, for all pages that use it.
   - Survives until the Web Application Server is restarted
   - Can be used for a database connection (or connection pool)

Example

```
<html>
<head>
<title>useBean Example</title>
</head>
<body>

<jsp:useBean id="date" class="java.util.Date" />
<p>The date/time is <%= date %>

</body>
</html>
```


jsp with javabeans example -
Java bean example in JSP
JavaServer Pages and JavaBe
JSP - JavaBeans
useBean Example
localhost:8080/AccessBean.jsp
The date/time is Thu Apr 19 19:02:00 IST 2012
start
useBean Example - G...
DD 121/5 (16.0 ov, K...
TextPad - C:\Progra...
learn1
JavaServerPage's [C...
7:03 PM

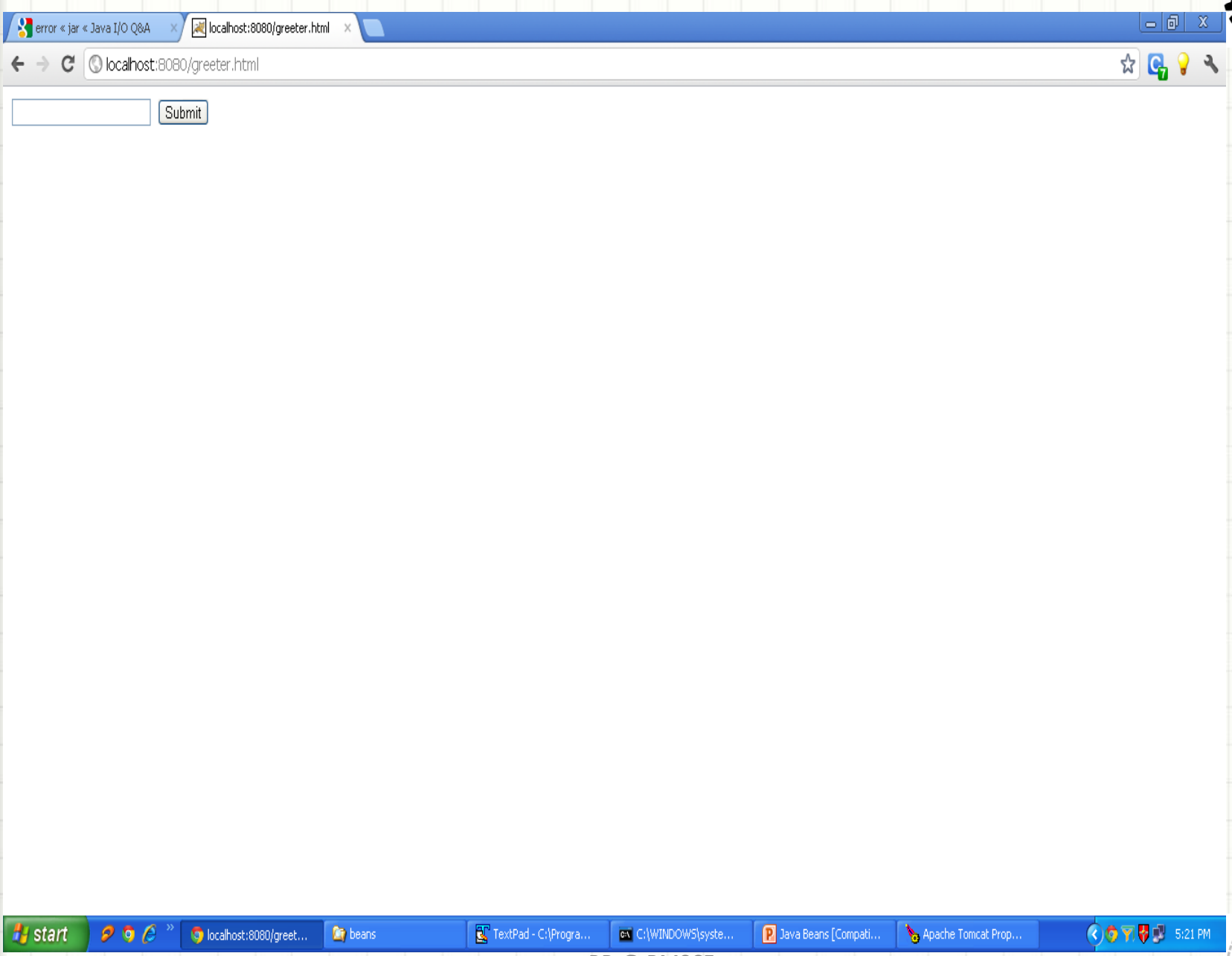
Example

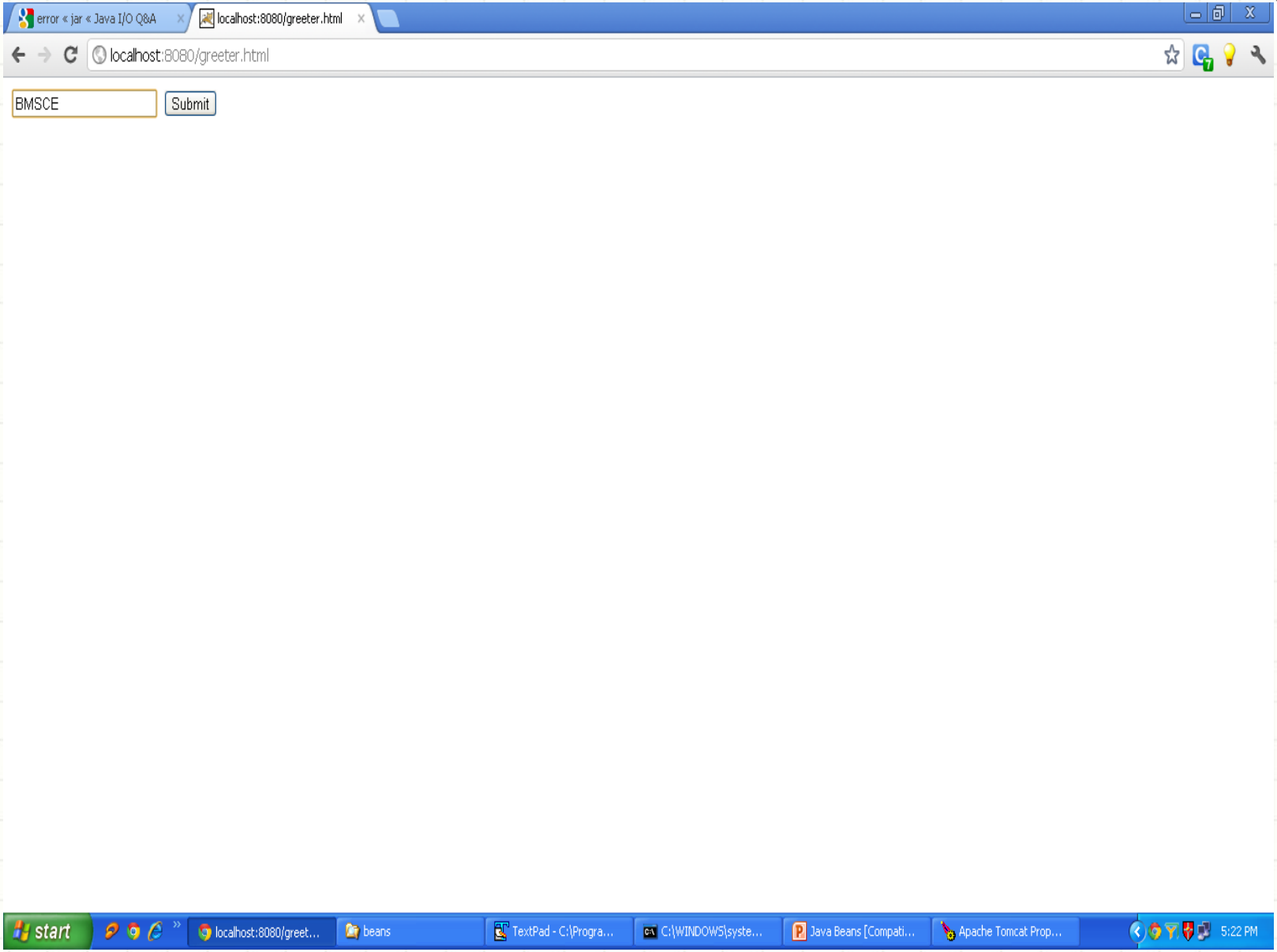
package ourbeans;

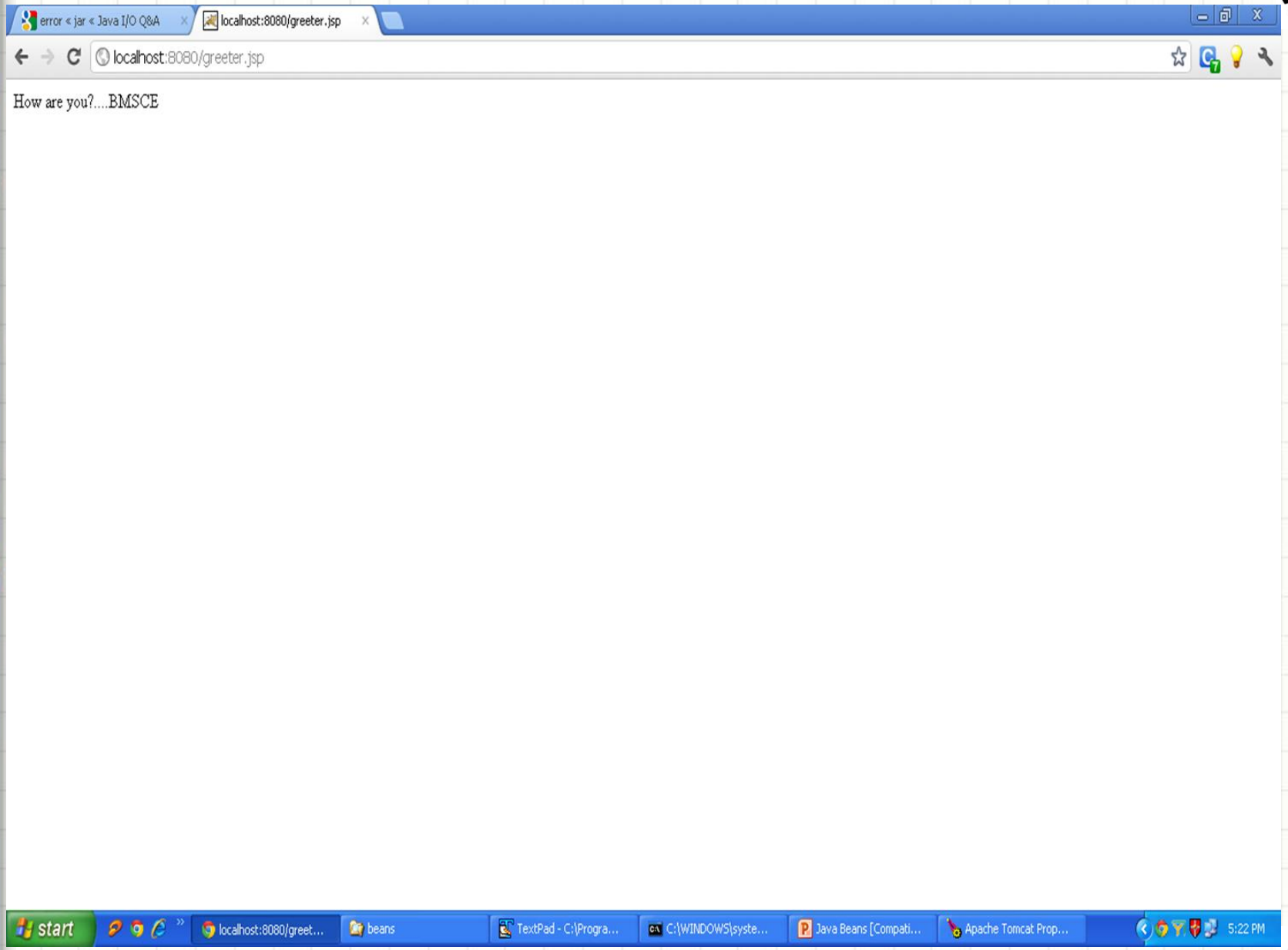
```
public  class greeterbean
{
 public greeterbean()   {  }
 public String  greetme(String s)
 {
 return   "How are you?...."+s;
 }
}
```

```
<html>
 <body>
  <form  method=post
action=greeter.jsp>
  <input   type=text  name='text1'>
  <input   type=submit>
   </form>      </body>      </html>
```

```
<html>
  <body>
   <jsp:useBean   id="greeter1"
class="ourbeans.greeterbean"  />
   <%
   String   s =
request.getParameter("text1");
    String   a = greeter1.greetme(s);
   out.println(a);
   %>
  </body>
  </html>
```


error « jar « Java I/O Q&A
localhost:8080/greeter.html
localhost:8080/greeter.html
Submit
start
localhost:8080/greet...
beans
TextPad - C:\Progra...
C:\WINDOWS\syste...
Java Beans [Compati...
Apache Tomcat Prop...
5:21 PM


error « jar « Java I/O Q&A
localhost:8080/greeter.html
localhost:8080/greeter.html
BMSCE
Submit
start
localhost:8080/greet...
beans
TextPad - C:\Progra...
C:\WINDOWS\syste...
Java Beans [Compati...
Apache Tomcat Prop...
5:22 PM


error « jar « Java I/O Q&A
localhost:8080/greeter.jsp
localhost:8080/greeter.jsp
How are you?....BMSCE
start
localhost:8080/greet...
beans
TextPad - C:\Progra...
C:\WINDOWS\syste...
Java Beans [Compati...
Apache Tomcat Prop...
5:22 PM

Example for Accessing Java Bean Properties

```
package learn1;
public class StudentsBean implements java.io.Serializable
{
  private String firstName = null;
  private String lastName = null;
  private int age = 0;

  public StudentsBean() {
  }
  public String getFirstName(){
    return firstName;
  }
  public String getLastName(){
    return lastName;
  }
  public void setFirstName(String firstName){
    this.firstName = firstName;
  }
  public void setLastName(String lastName){
    this.lastName = lastName;
  }
}
```

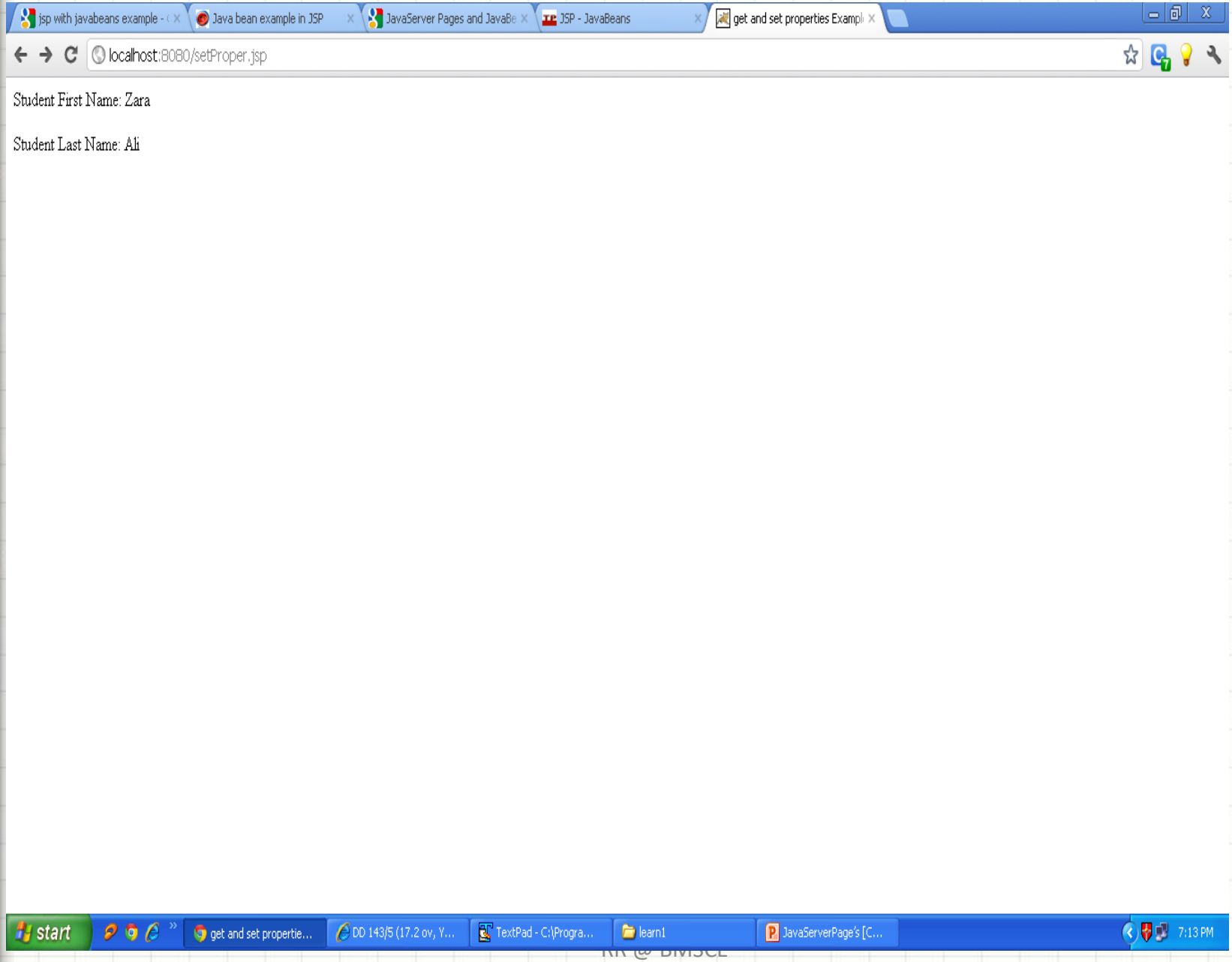
```
<html>
<head>
<title>get and set properties Example</title>
</head>
<body>

<jsp:useBean id="students" class="learn1.StudentsBean">
  <jsp:setProperty name="students" property="firstName" value="Zara"/>
  <jsp:setProperty name="students" property="lastName" value="Ali"/>

</jsp:useBean>

<p>Student First Name: <jsp:getProperty name="students"
property="firstName"/>
</p>
<p>Student Last Name: <jsp:getProperty name="students"
property="lastName"/>


</body>
</html>
```


jsp with javabeans example -
Java bean example in JSP
JavaServer Pages and JavaBe
JSP - JavaBeans
get and set properties Exampl
localhost:8080/setProper.jsp
Student First Name: Zara
Student Last Name: Ali
start
get and set propertie...
DD 143/5 (17.2 ov, Y...
TextPad - C:\Progra...
learn1
JavaServerPage's [C...
7:13 PM

taglib Directive (Cont..)

The **Java Server Pages** API allows to define custom JSP tags that look like HTML or XML tags and a tag library is a set of user-defined tags that implement custom behavior.

The taglib directive declares that your JSP page uses a set of custom tags, identifies the location of the library, and provides a means for identifying the custom tags in your JSP page.

The taglib directive follows the following syntax:

```
<%@ taglib uri="uri" prefix="prefixOfTag" >
```

Where the uri attribute value resolves to a location the container understands and the prefix attribute informs a container what bits of markup are custom actions.

You can write XML equivalent of the above syntax as follows:

```
<jsp:directive.taglib uri="uri" prefix="prefixOfTag" />
```

Check more detail related to taglib directive at Taglib Directive.

JSTL

Introduction

- The Java Server Pages Standard Tag Library (JSTL) is a collection of useful JSP tags which encapsulates core functionality common to many JSP applications.

- The JSTL tags can be classified, according to their functions, into following JSTL tag library groups that can be used when creating a JSP page:

  1. Core Tags
  2. Formatting tags
  3. SQL tags
  4. XML tags
  5. JSTL Functions

Core Tags

The core group of tags are the most frequently used JSTL tags. Following is the syntax to include JSTL Core library in your JSP:

```
<%@ taglib prefix="c"
        uri="http://java.sun.com/jsp/jstl/core" %>
```

There are following Core JSTL Tags:

Tag	Description
<c:out >	Like <%= ... >, but for expressions.
<c:set >	Sets the result of an expression evaluation in a 'scope'
<c:remove >	Removes a scoped variable (from a particular scope, if specified
<c:catch>	Catches any Throwable that occurs in its body and optionally exposes it.
<c:if>	Simple conditional tag which evalutes its body if the supplied condition is
<c:choose>	Simple conditional tag that establishes a context for mutually
<c:when>	Subtag of <choose> that includes its body if its condition evalutes to 'true'.
<c:otherwise >	Subtag of <choose> that follows <when> tags and runs only if all
<c:import>	Retrieves an absolute or relative URL and exposes its contents to either the page, a String in 'var', or a Reader in 'varReader'.
<c:forEach >	The basic iteration tag, accepting many different collection types and supporting subsetting and other functionality .
<c:forTokens>	Iterates over tokens, separated by the supplied delimeters.
<c:param>	Adds a parameter to a containing 'import' tag's URL.
<c:redirect >	Redirects to a new URL.

Installing JSTL Library

If you are using Apache Tomcat container then follow the following two simple steps:

- Download the binary distribution from Apache Standard Taglib and unpack the compressed file.
- To use the Standard Taglib from its Jakarta Taglibs distribution, simply copy the JAR files in the distribution's 'lib' directory to your application's webapps\ROOT\WEB-INF\lib directory.
- To use any of the libraries, you must include a <taglib> directive at the top of each JSP that uses the library.

QUESTIONS?

APPENDIX